PROGRAMACIÓN ANALÍTICA II

# Guía de estudio para el parcial

## Temas 1 a 6 + Práctico 1 + Práctico 2

S2 2026 | Docente: Camila Palomeque

Preparada a partir de los notebooks oficiales del curso, tus prácticos y el simulacro adjunto.

**Objetivo:** Que puedas leer esta guía de punta a punta, entender qué hace cada herramienta y después resolver el simulacro sin mirar las respuestas. Por eso esta guía NO incluye la solución del simulacro.

### Fuentes utilizadas en esta guía

- Buenas prácticas de código (Notebook).ipynb
- 2_Correctores de codigo (linting).ipynb
- 3_Funciones.ipynb
- Práctico 1 Buenas prácticas de código y Funciones + Practico1_RuggieroFranco.ipynb
- 4_Pandas - análisis básico y transformaciones.ipynb
- 5_Pandas - agrupaciones y combinaciones fuentes de datos.ipynb
- Práctico 2 Pandas (temas 4 y 5)
- 6_Visualizaciones (Matplotlib y Seaborn).ipynb
- Simulacro_Parcial_S2_2026.ipynb

# 0. Cómo está armado el parcial y cómo estudiar

El simulacro indica que se evalúan los temas 1 a 6. La parte teórica se entrega en papel y la parte práctica en un notebook. Algunas preguntas de múltiple opción pueden admitir más de una respuesta correcta y marcar una opción incorrecta puede restar puntos dentro de esa pregunta.

| Parte | Qué aparece en el simulacro | Qué tenés que saber hacer |
|---|---|---|
| Teórica | Múltiple opción + desarrollo | Leer código, completar huecos, detectar errores conceptuales y explicar en pocas frases. |
| Práctica | 2 ejercicios de análisis | Escribir funciones simples, auditar datos, agrupar, interpretar y elegir un gráfico correcto. |

**Observación del archivo:** La tabla inicial del simulacro dice 8 preguntas de múltiple opción, pero el notebook contiene P1 a P10, además de D1 y D2. Para estudiar conviene cubrir el contenido real de las celdas.

## Orden de estudio recomendado

1. Entender las diferencias conceptuales: return vs print, agg vs transform, merge vs concat, isna vs notna, promedio vs mediana.
2. Aprender los patrones cortos de código que se repiten en los prácticos.
3. Practicar completar código sin ejecutarlo: mirar qué objeto entra, qué método corresponde y qué debe devolver.
4. En la parte práctica: primero auditar, después transformar, después resumir y recién al final interpretar/graficar.

**Regla del curso:** La tabla o el gráfico no son el resultado: son la evidencia. Siempre hay que cerrar con una interpretación que responda la pregunta de negocio.

# 1. Tema 1 - Clean Code

La idea central del notebook es que el código se escribe una vez y se lee muchas veces. El objetivo es optimizar para la lectura, no demostrar que se puede escribir algo complicado.

## Lo que tenés que reconocer

| Principio | Qué significa en este curso | Señal de alerta |
|---|---|---|
| Nombres descriptivos | Variables y funciones deben revelar la intención. | proc, f, t, n cuando podrían tener nombres claros. |
| Funciones cortas | Una responsabilidad por función. | Una función filtra, calcula, imprime y valida todo junto. |
| return | La función calcula y devuelve; el resultado se puede reutilizar. | Usar print como sustituto de return. |
| Guard clauses / lógica plana | Resolver casos especiales temprano para evitar anidación. | if dentro de if dentro de if. |
| DRY | No repetir la misma lógica o fórmula. | La misma fórmula copiada en varios lugares. |
| Constantes con nombre | Los valores de negocio se declaran una vez. | 0.20, 1000 o listas de categorías repetidas sin explicación. |
| Docstring | Explica qué hace una función cuando el nombre no alcanza. | Comentarios que solo repiten cada línea. |

## Ejemplo típico: unir condiciones y mejorar el nombre

```
def sumar_facturas_activas_en_pesos(facturas):
    total = 0
    for factura in facturas:
        if (factura["estado"] == "Activo"
                and factura["monto"] > 0
                and factura["moneda"] == "UYU"):
            total += factura["monto"]
    return total
```

**Idea de parcial:** Si el código funciona pero el nombre no dice qué hace, o tiene tres if anidados que pueden expresarse con una condición clara, sigue siendo un problema de Clean Code.

## print vs return

```
def calcular_total(montos):
    total = sum(montos)
    return total

resultado = calcular_total([10, 20, 30])
print(resultado)
```

`return` entrega el valor a quien llamó a la función. `print` solo lo muestra. En los prácticos la regla fue: las funciones calculan y devuelven; los `print` van afuera.

## Constantes y números mágicos

```
UMBRAL_TASA_AHORRO = 0.20
CATEGORIAS_ESENCIALES = ["Vivienda", "Salud", "Supermercado"]
```

La ventaja no es matemática: el lector entiende qué significa el 0.20 y, si cambia la política, el valor se modifica en un solo lugar.

## Qué NO hace falta meter de más

El material de linting aclara que en este curso no se espera usar anotaciones de tipo en las funciones. Para el parcial conviene escribir firmas simples y legibles, como se hizo en los prácticos.

# 2. Tema 2 - Linting: Black, flake8 y pylint

Linting es análisis estático: las herramientas leen el archivo sin ejecutarlo y detectan problemas de formato, errores probables y cuestiones de diseño.

**Orden que hay que saber:** 1) black formatea. 2) flake8 revisa lo que quedó y detecta errores/estilo. 3) pylint evalúa diseño y da un puntaje.

| Herramienta | Para qué sirve | Qué NO hace |
|---|---|---|
| black | Reescribe el formato de manera automática. | No arregla lógica, no renombra y no borra imports sin usar. |
| flake8 | Detecta problemas de estilo y bugs probables. | No rediseña el código ni decide la mejor arquitectura. |
| pylint | Mira diseño, convenciones y da un puntaje sobre 10. | El puntaje no es un porcentaje de que el programa funcione. |

## PEP 8 que sí aparece en el curso

- snake_case para variables y funciones: calcular_margen, tasa_anual.
- Imports arriba y agrupados.
- Espacios alrededor de operadores: margen = ventas - costo.
- Dos líneas en blanco entre funciones de nivel superior.
- Límite de línea usado en clase: 88 caracteres cuando se corre flake8/black.

## Cómo leer flake8

| Prefijo | Qué indica | Prioridad práctica |
|---|---|---|
| F | Problemas de pyflakes: nombres inexistentes, imports/variables problemáticos. | Primero. Pueden ser bugs reales. |
| E / W | Errores y warnings de estilo. | Muchos los resuelve black; otros requieren ajuste manual. |
| C | Complejidad excesiva. | Revisar estructura de la función. |

## Cómo leer pylint

| Prefijo | Idea |
|---|---|
| C | Convención: nombres, docstrings, estilo. |
| R | Refactorización sugerida. |
| W | Advertencia: algo raro, no necesariamente fatal. |
| E | Error probable. |

```
python -m black --line-length 88 archivo.py
python -m flake8 --max-line-length=88 archivo.py
```

```
python -m pylint archivo.py
```

**Trampa frecuente:** Black puede dejar el archivo lindo y el programa seguir roto. Formato correcto no significa lógica correcta.

# 3. Tema 3 - Funciones, listas, diccionarios y bucles

Este tema aparece muy fuerte en el Práctico 1 y también en la parte práctica del simulacro. Hay que poder leer y escribir funciones cortas sin depender de Pandas.

## Anatomía de una función

```
def aplicar_descuento(precio, unidades, descuento=0.10):
    total = precio * unidades * (1 - descuento)
    return total
```

- `precio` y `unidades` son parámetros obligatorios.
- `descuento=0.10` es un parámetro con valor por defecto.
- Un parámetro sin valor por defecto no puede colocarse después de uno que sí tiene valor por defecto.
- Se pueden usar argumentos con nombre: aplicar_descuento(unidades=2, precio=50).

## Retorno múltiple y desempaquetado

```
def calcular_margen(facturacion, costo_total):
    margen = facturacion - costo_total
    margen_pct = margen / facturacion * 100
    return margen, margen_pct

margen_pesos, margen_porcentaje = calcular_margen(68400, 53800)
```

Si una función devuelve dos valores, se pueden recibir en dos variables en el mismo orden.

## Patrón 1: filtrar una lista

```
def filtrar_gastos(transacciones):
    gastos = []
    for transaccion in transacciones:
        if transaccion["monto"] < 0:
            gastos.append(transaccion)
    return gastos
```

## Patrón 2: acumulador

```
def sumar_montos(transacciones):
    monto_total = 0.0
    for transaccion in transacciones:
        monto_total += transaccion["monto"]
    return monto_total
```

## Patrón 3: contar por categoría con diccionario

```
def contar_por_categoria(transacciones):
    conteo = {}
    for transaccion in transacciones:
        categoria = transaccion["categoria"]
        if categoria in conteo:
            conteo[categoria] += 1
        else:
            conteo[categoria] = 1
```

```
    return conteo
```

## Patrón 4: resumen mensual con diccionario anidado

```
def resumir_por_mes(transacciones):
    resumen = {}
    for transaccion in transacciones:
        mes = transaccion["fecha"][:7]
        monto = transaccion["monto"]

        if mes not in resumen:
            resumen[mes] = {"ingresos": 0.0, "gastos": 0.0}

        if monto > 0:
            resumen[mes]["ingresos"] += monto
        else:
            resumen[mes]["gastos"] += monto

    for mes in resumen:
        resumen[mes]["balance"] = (
            resumen[mes]["ingresos"] + resumen[mes]["gastos"]
        )
    return resumen
```

**Por qué se suma el gasto en el balance:** En el Práctico 1 los gastos están guardados con signo negativo. Por eso balance = ingresos + gastos.

## Validaciones básicas

```
if total_ingresos == 0:
    print("No se puede calcular la tasa de ahorro sin ingresos")
    return None
```

La idea es evitar divisiones por cero o resultados engañosos. El simulacro también puede pedir una función que clasifique según reglas con `if` / `elif` / `else`.

## Listas vs diccionarios

| Estructura | Cuándo se usa | Ejemplo |
|---|---|---|
| Lista | Colección ordenada de elementos. | gastos = [] |
| Diccionario | Pares clave-valor; natural para representar una fila o un resumen. | {"categoria": "Ocio", "monto": -800} |
| Lista de diccionarios | Patrón central del Práctico 1 antes de Pandas. | [{"fecha": ..., "monto": ...}, ...] |

# 4. Práctico 1 - Qué tenés que dominar

El Práctico 1 evalúa Temas 1, 2 y 3. Lo importante no es memorizar tus respuestas, sino reconocer los patrones que usaste.

| Parte | Qué se practicó | Patrón que puede volver en el parcial |
|---|---|---|
| Clean Code | Refactorizar una función larga que calculaba gasto promedio, gasto esencial, ingresos y tasa de ahorro. | Separar responsabilidades, constantes con nombre, funciones con verbo. |
| Linting | Limpiar presupuesto_sucio.py. | Black -> flake8 -> pylint; distinguir formato, bug y diseño. |
| Carga CSV | csv.DictReader + try/except. | Convertir monto y contar descartadas. |
| Funciones | filtrar_gastos, contar_por_categoria, clasificar, sumar por categoría. | for + if + acumuladores + diccionarios. |
| Resumen mensual | Diccionario anidado por AAAA-MM. | Crear grupo manualmente y acumular ingresos/gastos. |

## Carga de CSV sin Pandas: qué hace cada parte

```
with open(ruta_archivo, encoding="utf-8") as archivo:
    for fila in csv.DictReader(archivo):
        try:
            fila["monto"] = float(fila["monto"])
            transacciones.append(fila)
        except (ValueError, TypeError):
            descartadas += 1
```

El `try` intenta convertir el monto. Si no se puede, el `except` evita que el programa se corte y cuenta la fila descartada.

## Tasa de ahorro

```
tasa_ahorro = (total_ingresos + total_gastos) / total_ingresos
```

En estos datos `total_gastos` es negativo. Si se trabaja con gastos convertidos a positivo, la fórmula tendría que cambiar. Antes de aplicar una fórmula, mirá cómo están representados los datos.

**Lección del Práctico 1:** Antes de programar, definí qué entra, qué tiene que devolver la función y qué signo/formato tienen los datos. Muchos errores no son de sintaxis: son de interpretación.

# 5. Tema 4 - Pandas: cargar, explorar, limpiar y transformar

Tema 4 marca el cambio desde listas de diccionarios a DataFrames. La secuencia típica del curso es: cargar -> auditar -> decidir -> transformar -> interpretar.

## Carga y análisis exploratorio básico

```
ventas = pd.read_csv(
    "ventas_ecommerce.csv",
    parse_dates=["fecha"]
)

print(ventas.shape)
display(ventas.dtypes)
display(ventas.isna().sum())
display(ventas[["unidades", "precio_unitario"]].describe().round(2))
```

| Método | Pregunta que responde |
|---|---|
| shape | ¿Cuántas filas y columnas hay? |
| dtypes | ¿Cada columna tiene el tipo correcto? |
| isna().sum() | ¿Cuántos faltantes hay? |
| describe() | ¿Qué dicen media, desvío, mínimos, cuartiles y máximos? |
| min() / max() | ¿Cuál es el rango, por ejemplo de fechas? |

**Mentalidad de auditoría:** Si `describe()` muestra unidades negativas o un costo que no debería ser negativo, no hay que corregir automáticamente. Primero se localizan las filas y se entiende si es error, excepción o regla del negocio.

## Filtros

```
# Una condición
perdedoras = ventas[ventas["margen_neto"] < 0]

# Dos condiciones: cada condición va entre paréntesis
ejemplo = ventas[
    (ventas["unidades"] > 0)
    & (ventas["canal"] == "Marketplace")
]

# Elegir filas y columnas
ventas.loc[ventas["margen_neto"] < 0, ["id_orden", "margen_neto"]]
```

## Nulos

```
ventas["costo_envio"].isna()     # True donde falta
ventas["costo_envio"].notna()    # True donde existe
ventas["costo_envio"].fillna(0)  # reemplaza faltantes con 0
```

`isna()` y `notna()` devuelven booleanos. `fillna()` sí modifica/reemplaza los valores faltantes en la serie resultante.

## np.where

```
ventas["estado"] = np.where(
    ventas["margen_neto"] < 0,
    "Pierde",
    "No pierde"
)
```

Se lee: si la condición es True pone el primer valor; si es False, pone el segundo.

## Duplicados: dos preguntas distintas

```
ventas.duplicated().sum()
ventas.duplicated(subset="id_orden").sum()
ventas_limpias = ventas.drop_duplicates().copy()
```

- `duplicated()` sin subset pregunta por filas 100% iguales.
- `duplicated(subset="id_orden")` pregunta si el identificador aparece más de una vez, aunque el resto de la fila pueda diferir.
- `.copy()` evita trabajar sobre una vista ambigua de otra tabla y aparece explícitamente en el material.

## Conteos, proporciones, orden y top

```
ventas["categoria"].value_counts()
ventas["categoria"].value_counts(normalize=True)
ventas.sort_values("facturacion", ascending=False)
ventas.nlargest(10, "facturacion")
```

## Fechas

```
ventas["mes"] = ventas["fecha"].dt.month
ventas["periodo"] = ventas["fecha"].dt.strftime("%Y-%m")
ventas["dias"] = (ventas["fecha_fin"] - ventas["fecha_inicio"]).dt.days
```

## Percentiles y cortes

```
p90 = ventas["facturacion"].quantile(0.90)

ventas["tramo_ticket"] = pd.qcut(
    ventas["facturacion"],
    5,
    labels=["baja", "media-baja", "media", "media-alta", "alta"]
)
```

`qcut` corta buscando grupos con cantidades similares de observaciones. `cut` usa cortes por valor definidos por negocio. El Tema 4 remarca que sirven para cosas distintas.

# 6. Caso A - Banco Litoral: por qué sirve para el parcial

Este caso fue la aplicación de Tema 4 que te tocó. Es útil porque mezcla calidad de datos, columnas calculadas, clasificación e interpretación económica.

## Problemas de datos que aparecieron

- Duplicados exactos: había que medirlos y decidir si eliminarlos.
- `tasa_activa_anual` faltante en clientes sin préstamo: el faltante era estructural, no necesariamente un error.
- Saldos de depósito negativos: valor que merece revisión porque el descubierto debería representarse de otra manera según el caso.

## Ingreso anual por cliente

```
TASA_TRANSFERENCIA = 9.5

margen_prestamo = saldo_prestamo * (tasa_activa - TASA_TRANSFERENCIA) / 100
margen_deposito = saldo_deposito * (TASA_TRANSFERENCIA - tasa_pasiva) / 100
comisiones_anuales = comisiones_mes * 12

ingreso_anual = margen_prestamo + margen_deposito + comisiones_anuales
```

## Clasificación por valor

```
tramos = []
for ingreso in clientes_limpios["ingreso_anual"]:
    if ingreso < 0:
        tramos.append("Destruye valor")
    elif ingreso < 50_000:
        tramos.append("Marginal")
    elif ingreso < 500_000:
        tramos.append("Rentable")
    else:
        tramos.append("Estratégico")

clientes_limpios["tramo_valor"] = tramos
```

**Lectura importante:** Un faltante puede tener una explicación del proceso que genera los datos. El curso insiste en investigar el patrón antes de borrar o imputar.

# 7. Tema 5 - groupby, agg, transform, merge y concat

Este es uno de los bloques más importantes del parcial. El simulacro pregunta directamente por `groupby`, `agg`, `transform` y la diferencia entre `merge` y `concat`.

## groupby + una métrica

```
facturas.groupby("linea_producto")["margen"].sum()
```

El resultado tiene una fila por grupo. La columna usada para agrupar queda como índice hasta que se use `reset_index()`.

## Agregaciones con nombre: .agg()

```
resumen = facturas.groupby("linea_producto").agg(
    cantidad=("id_factura", "count"),
    facturacion=("facturacion", "sum"),
    descuento_promedio=("descuento", "mean"),
    costo_mediano=("costo_envio", "median")
)

resumen = resumen.sort_values("facturacion", ascending=False)
```

**Trampa de examen:** Si la pregunta pide descuento promedio, la agregación debe ser `mean`, no `sum`. Y `head(3)` no significa "los tres mayores" si antes no ordenaste por la métrica correcta.

## agg vs transform

| | agg | transform |
|---|---|---|
| Qué devuelve | Un resumen: una fila por grupo. | Una serie alineada con todas las filas originales. |
| Cuándo usar | Cuando querés una tabla resumen. | Cuando cada fila necesita conocer una métrica de su grupo. |
| Ejemplo | Total facturado por categoría. | Total de la categoría repetido en cada orden. |

```
ventas["facturacion_categoria"] = (
    ventas.groupby("categoria")["facturacion"].transform("sum")
)

ventas["participacion_en_categoria_pct"] = (
    ventas["facturacion"] / ventas["facturacion_categoria"] * 100
)
```

## reset_index()

```
resumen = ventas.groupby("categoria")["facturacion"].sum().reset_index()
```

Sirve especialmente cuando después se quiere cruzar el resumen con otra tabla.

## merge

```
resultado = izquierda.merge(
    derecha,
    on="clave",
    how="left"
)
```

| how | Qué conserva |
|---|---|
| left | Todas las filas de la tabla izquierda, aunque no encuentren pareja. |
| inner | Solo las claves que existen en ambas tablas. |

En el curso se insiste en verificar cuántas filas entran y cuántas salen del merge, porque una clave duplicada del lado derecho puede inflar el número de filas.

## concat

```
todo = pd.concat([enero, febrero], ignore_index=True)
```

`concat` apila tablas con la misma estructura. No busca una clave para relacionarlas. Esa es la diferencia esencial con `merge`.

## Otras herramientas del Tema 5

| Necesidad | Patrón |
|---|---|
| Agrupar por dos columnas | tabla.groupby(["grupo", "subgrupo"])["monto"].mean() |
| Valores distintos | tabla.groupby("grupo")["cliente"].nunique() |
| Tabla cruzada porcentual | pd.crosstab(tabla["a"], tabla["b"], normalize="index") |
| Suma acumulada relativa | serie.cumsum() / serie.sum() |
| Ordenar | resumen.sort_values("total", ascending=False) |

# 8. Práctico 2 - Mapa completo de lo que hiciste

El Práctico 2 reúne los Temas 4 y 5 y agrega dos herramientas temporales que la propia consigna indica: `pct_change()` y `rolling()`.

## 1.1 - Exploración

- Cargar con tipos correctos y `parse_dates`.
- Revisar `shape`, `dtypes`, nulos, rango de fechas y `describe()`.
- Interpretar qué representa una fila y si el último mes/año está completo.
- Detectar valores que merecen revisión, no corregirlos automáticamente.

## 1.2 - Columnas calculadas

```
MARGEN_BRUTO_DEL_AREA_COMERCIAL = 0.35

ventas["facturacion"] = (
   ventas["unidades"] * ventas["precio_unitario"]
   * (1 - ventas["descuento"])
)
ventas["margen_bruto"] = ventas["facturacion"] * MARGEN_BRUTO_DEL_AREA_COMERCIAL
ventas["margen_neto"] = ventas["margen_bruto"] - ventas["costo_envio"]
ventas["periodo"] = ventas["fecha"].dt.strftime("%Y-%m")
```

La interpretación pedía separar dos explicaciones posibles de los márgenes negativos: órdenes chicas vs flete caro. La forma correcta es comparar números de ambos grupos, no quedarse con una impresión.

## 2.1 - Auditoría de calidad

```
cantidad_nulos = ventas_erp.isna().sum()
porcentaje_nulos = cantidad_nulos / len(ventas_erp) * 100

duplicados_exactos = ventas_erp.duplicated().sum()
id_repetidos = ventas_erp.duplicated(subset="id_orden").sum()

auditoria = ventas_erp.copy()
auditoria["flete_faltante"] = auditoria["costo_envio"].isna()

faltantes_canal = auditoria.groupby("canal")["flete_faltante"].mean() * 100
```

El promedio de una columna booleana funciona como proporción porque True se trata como 1 y False como 0.

**Nulos que se propagan:** Si `margen_neto = margen_bruto - costo_envio` y `costo_envio` es NaN, el margen neto también queda NaN. Después `sum()` ignora esos NaN y puede dejar órdenes completas fuera del total sin avisar.

## 2.2 - Decidir y medir impacto

- La decisión de limpieza tiene que quedar escrita y justificada.
- Puede ser eliminar, imputar o incluso no hacer nada, si se justifica.
- Siempre medir el antes y después: filas, facturación total y margen neto total.

### 3.1 - Resumen por categoría

```
resumen_categoria = ventas.groupby("categoria").agg(
    ordenes=("id_orden", "count"),
    unidades=("unidades", "sum"),
    facturacion_total=("facturacion", "sum"),
    ticket_promedio=("facturacion", "mean"),
    ticket_mediano=("facturacion", "median"),
    descuento_promedio=("descuento", "mean")
)

resumen_categoria["brecha_pct"] = (
    resumen_categoria["ticket_promedio"]
    / resumen_categoria["ticket_mediano"] - 1
) * 100
```

Promedio por encima de mediana sugiere que valores altos empujan el promedio. Si se quiere representar una orden típica y hay asimetría, la mediana suele ser más robusta.

### 3.2 - Concentración con transform

```
ventas["facturacion_categoria"] = (
    ventas.groupby("categoria")["facturacion"].transform("sum")
)
ventas["participacion_en_categoria_pct"] = (
    ventas["facturacion"] / ventas["facturacion_categoria"] * 100
)
```

Después se ordenan las órdenes por facturación, se toman las 10 mayores de cada categoría y se suma su participación para estudiar concentración.

### 4.1 - Serie mensual

```
serie_mensual = (
    ventas.set_index("fecha")["facturacion"]
    .resample("ME")
    .sum()
    .reset_index()
)

serie_mensual["variacion_mensual_pct"] = (
    serie_mensual["facturacion"].pct_change() * 100
)
serie_mensual["variacion_interanual_pct"] = (
    serie_mensual["facturacion"].pct_change(12) * 100
)
serie_mensual["media_movil_3_meses"] = (
    serie_mensual["facturacion"].rolling(3).mean()
)
```

**Por qué interanual:** En un negocio con estacionalidad, comparar junio con mayo puede mezclar gestión con patrón estacional. La variación interanual compara junio contra junio.

## 4.2 - Merge con contexto macro

```
macro = pd.read_csv(
    "indicadores_macro_uy.csv",
    parse_dates=["fecha"]
)

serie_macro = serie_mensual.merge(
    macro,
    on="fecha",
    how="left"
)
```

La consigna pide usar actividad, inflación, tipo de cambio y salario real para evaluar si una caída parece venir del negocio o del contexto. La conclusión debe ser prudente: mirar si los indicadores se mueven en una dirección compatible con la caída, sin afirmar causalidad que los datos no demuestran.

# 9. Tema 6 - Visualizaciones con Matplotlib y Seaborn

El objetivo explícito del Tema 6 es elegir el gráfico que responde a la pregunta, no el que sale más rápido. El estilo de clase usa la interfaz `fig, ax`.

## Esqueleto que conviene memorizar

```
fig, ax = plt.subplots(figsize=(9, 4))

# ax.plot(...), ax.barh(...), ax.scatter(...), etc.

ax.set_title("Título", loc="left")
ax.set_xlabel("Eje x")
ax.set_ylabel("Eje y")
plt.tight_layout()
plt.show()
```

## Qué gráfico responde a qué pregunta

| Pregunta | Gráfico |
|---|---|
| ¿Cómo evolucionó esto en el tiempo? | Línea |
| ¿Cuánto tiene cada categoría? | Barras horizontales ordenadas |
| ¿Cómo se compone el total de cada categoría? | Barras apiladas |
| ¿Cómo se comparan dos series por categoría? | Barras agrupadas |
| ¿Cómo se reparten los valores de una variable? | Histograma |
| ¿Qué tan dispersa es la variable dentro de cada grupo? | Boxplot |
| ¿Estas dos variables numéricas van juntas? | Dispersión / scatter |
| ¿Qué variables se mueven juntas entre muchas? | Heatmap de correlación |
| ¿Cuánto es un solo número clave? | Ningún gráfico: escribir el número |

## Línea

```
fig, ax = plt.subplots(figsize=(9, 4))
ax.plot(serie.index, serie.values)
ax.set_title("Evolución mensual", loc="left")
ax.set_xlabel("Mes")
ax.set_ylabel("Facturación")
plt.tight_layout()
plt.show()
```

Solo tiene sentido cuando el eje x tiene orden natural, como días, meses o años.

## Barras horizontales

```
fig, ax = plt.subplots(figsize=(9, 4))
ax.barh(resumen["linea_producto"], resumen["facturacion"])
ax.set_title("Facturación por línea de producto", loc="left")
ax.set_xlabel("Facturación")
```

```
ax.set_ylabel("Línea de producto")
plt.tight_layout()
plt.show()
```

**Orden de barh:** Primero va la variable categórica del eje y y después el valor numérico del eje x.

## Histograma

```
fig, ax = plt.subplots(figsize=(9, 4))
ax.hist(ventas["facturacion"], bins=40)
ax.set_xlabel("Facturación")
ax.set_ylabel("Cantidad de órdenes")
plt.show()
```

Sirve para mirar la distribución de UNA variable numérica: forma, asimetría, concentración y valores extremos.

## Boxplot

El material lo usa para comparar dispersión entre grupos. Es especialmente útil cuando una barra de promedios escondería variabilidad y outliers.

## Scatter

```
fig, ax = plt.subplots(figsize=(9, 4))
ax.scatter(ventas["facturacion"], ventas["costo_envio"], alpha=0.5)
ax.set_xlabel("Facturación")
ax.set_ylabel("Costo de envío")
plt.show()
```

Si la pregunta es cómo se relacionan dos variables numéricas, scatter es el gráfico natural. El simulacro práctico plantea exactamente ese tipo de decisión.

## Seaborn

El tema muestra `sns.barplot`, `sns.boxplot` y `sns.heatmap`. Para el parcial, lo esencial es reconocer qué pregunta responde cada gráfico y poder completar el código base que te den.

## Tres problemas visuales que el tema quiere evitar

- Eje truncado en barras: puede exagerar diferencias porque la longitud de la barra deja de representar correctamente el valor.
- Elegir un gráfico que no corresponde a la pregunta.
- Sobrecargar el gráfico en vez de destacar solo la información que importa.

## 10. Qué te está anticipando el simulacro

Sin resolverlo todavía, el simulacro permite identificar con bastante precisión el tipo de destreza que se espera.

| Bloque | Qué te puede pedir |
|---|---|
| Clean Code | Elegir afirmaciones correctas sobre nombres, anidación y return. |
| Linting | Reconocer el orden Black -> flake8 -> pylint. |
| Funciones | Completar un return múltiple y razonar sobre parámetros por defecto. |
| Pandas básico | Interpretar describe, completar isna/notna, auditar datos. |
| Agrupaciones | Leer groupby, corregir una agg, ordenar antes de head, usar transform. |
| Uniones | Elegir concat cuando se apilan meses; merge cuando se cruzan fuentes por clave. |
| Visualización | Elegir gráfico por pregunta y completar `fig, ax` / `barh` / scatter. |
| Práctica | Filtrar márgenes negativos, escribir una función, auditar calidad, resumir con agg y graficar una relación. |

**Muy importante:** En el simulacro hay código que corre sin errores pero responde mal la pregunta. Eso significa que no alcanza con saber sintaxis: tenés que verificar si `sum`, `mean`, `head`, el orden y el gráfico elegido contestan realmente lo pedido.

### Método para completar un hueco de código

5. Leé la pregunta en español antes que las opciones.
6. Identificá el tipo de objeto: lista, diccionario, Series, DataFrame, figura/ax.
7. Preguntate qué debe quedar después de la línea: un valor, una serie, una columna nueva o una tabla resumen.
8. Descartá opciones que tienen el método correcto pero la dirección lógica invertida (por ejemplo isna vs notna).
9. Si hay `head(3)`, comprobá si antes hubo un `sort_values` por la métrica que interesa.

## 11. Diferencias que NO podés confundir

| A | B | Diferencia clave |
|---|---|---|
| print | return | print muestra; return entrega un valor reutilizable. |
| sum | mean | sum totaliza; mean promedia. La pregunta de negocio decide cuál corresponde. |
| isna() | notna() | isna es True si falta; notna es True si existe. |
| duplicated() | duplicated(subset="id") | Fila completa repetida vs identificador repetido. |
| agg | transform | agg reduce a una fila por grupo; transform conserva las filas originales. |
| merge | concat | merge relaciona por clave; concat apila estructuras compatibles. |
| left | inner | left conserva toda la izquierda; inner conserva solo coincidencias. |
| mean | median | La media se mueve con extremos; la mediana es más robusta. |
| pct_change() | pct_change(12) | Cambio contra período anterior vs contra 12 períodos atrás. |
| línea | scatter | Línea: evolución temporal; scatter: relación entre dos variables numéricas. |
| histograma | boxplot | Histograma: distribución de una variable; boxplot: comparar dispersión entre grupos. |

# 12. Plantillas cortas de código que deberías reconocer de memoria

## Filtrar

```
filtrado = tabla[tabla["monto"] < 0]
```

## Dos condiciones

```
filtrado = tabla[
    (tabla["a"] > 0) & (tabla["b"] == "Si")
]
```

## Nulos y duplicados

```
tabla.isna().sum()
tabla.duplicated().sum()
tabla.duplicated(subset="id").sum()
tabla_limpia = tabla.drop_duplicates().copy()
```

## Columna condicional

```
tabla["estado"] = np.where(condicion, "Si", "No")
```

## Resumen por grupo

```
resumen = tabla.groupby("grupo").agg(
    cantidad=("id", "count"),
    total=("monto", "sum"),
    promedio=("monto", "mean")
)
resumen = resumen.sort_values("total", ascending=False)
```

## Transform

```
tabla["total_grupo"] = (
    tabla.groupby("grupo")["monto"].transform("sum")
)
```

## Merge y concat

```
cruce = izquierda.merge(derecha, on="clave", how="left")
apilada = pd.concat([tabla1, tabla2], ignore_index=True)
```

## Top N

```
top = tabla.nlargest(10, "monto")
```

```
# o
ordenada = tabla.sort_values("monto", ascending=False).head(10)
```

## Serie temporal

```
mensual = tabla.set_index("fecha")["facturacion"].resample("ME").sum()
variacion = mensual.pct_change() * 100
interanual = mensual.pct_change(12) * 100
media_movil = mensual.rolling(3).mean()
```

## Gráfico de relación entre dos numéricas

```
fig, ax = plt.subplots(figsize=(9, 4))
ax.scatter(tabla["x"], tabla["y"], alpha=0.5)
ax.set_title("Título", loc="left")
ax.set_xlabel("X")
ax.set_ylabel("Y")
plt.tight_layout()
plt.show()
```

## 13. Checklist final antes de hacer el simulacro

Si podés explicar y escribir sin mirar todo lo de esta lista, estás en condiciones de sentarte con el simulacro.

☐ Explico por qué un nombre descriptivo y una función corta mejoran el código.
☐ Sé la diferencia entre print y return.
☐ Sé devolver dos valores y desempaquetarlos.
☐ Sé cómo funcionan parámetros por defecto y argumentos con nombre.
☐ Recuerdo el orden Black -> flake8 -> pylint y qué hace cada uno.
☐ Puedo escribir un for con if para filtrar y un acumulador para sumar.
☐ Puedo contar por categoría usando un diccionario.
☐ Sé leer shape, dtypes, isna y describe.
☐ Sé filtrar con una y dos condiciones en Pandas.
☐ Sé distinguir fila duplicada de ID repetido.
☐ Sé usar fillna, isna, notna y np.where.
☐ Sé construir una agg con count, sum, mean, median, min, max, std o nunique.
☐ Sé explicar agg vs transform.
☐ Sé explicar merge vs concat y left vs inner.
☐ Sé ordenar antes de usar head cuando busco los mayores.
☐ Sé para qué sirven pct_change, pct_change(12) y rolling(3).
☐ Sé elegir línea, barras, histograma, boxplot o scatter según la pregunta.
☐ Sé completar el esqueleto fig, ax con título y etiquetas.
☐ En una interpretación, respondo la pregunta y cito números de la tabla/gráfico.

**Siguiente paso:** Leé esta guía una vez completa. Después hacemos juntos el simulacro desde P1, sin mirar una clave de respuestas: primero respondés vos y yo te corrijo y explico cada elección.

# 14. Hoja de repaso de 5 minutos

| Si te piden... | Pensá en... |
|---|---|
| Devolver un resultado | return |
| Mostrar algo | print / display |
| Formatear código | black |
| Detectar errores/estilo | flake8 |
| Diseño/puntaje | pylint |
| Faltantes | isna / notna / fillna |
| Duplicados | duplicated / drop_duplicates |
| Clasificación binaria | np.where |
| Resumen por grupo | groupby + agg |
| Métrica de grupo en cada fila | groupby + transform |
| Cruzar por clave | merge |
| Apilar tablas | concat |
| Mayores N | nlargest o sort_values + head |
| Cambio temporal | pct_change |
| Cambio mismo mes año anterior | pct_change(12) |
| Suavizar últimos 3 meses | rolling(3).mean() |
| Tiempo | línea |
| Categorías | barras / barh |
| Distribución | histograma |
| Dispersión por grupos | boxplot |
| Relación entre 2 numéricas | scatter |

## Recordatorio final

No te apures a escribir. En este parcial muchas trampas son semánticas: el método existe y el código corre, pero responde otra pregunta. Leé qué quiere medir la consigna, revisá signos/nulos/grupos y recién después elegí la operación.